

Teaching Exam

Subject – Computer Science

**Computational Thinking and
Programming**



Lecture No –6

By–Satyendra Chaudhary Sir

Topics to be Covered



Topic

while for loop } ✓
nested loop
Dictionaries ✓
Import modules x

DICTIONARY

A dictionary is mutable and is another container type that can store any number of Python objects, including other container types.

Dictionaries consist of pairs (called items) of keys and their corresponding values.

$\text{dict} = \{ \text{'Alice'} : \text{'2341'}, \text{'Beth'} : \text{'9102'}, \text{'Cecil'} : \text{'3258'} \}$

Handwritten annotations:
 - Above the dictionary: $\text{dict} = [\overset{-3}{\text{'2341'}}, \overset{-2}{\text{'9102'}}, \overset{-1}{\text{'3258'}}]$ with values written to the left.
 - Below the dictionary: $\text{dict} = \{ \overset{\text{key}}{\text{'Alice'}} : \overset{\text{value}}{\text{'2341'}}, \text{'Beth'} : \text{'9102'}, \text{'Cecil'} : \text{'3258'} \}$ with key and value written below the first item.
 - Between the two lists: $\overset{0}{\text{'2341'}} \rightarrow \text{keys}$
 - To the right: $\text{dict}[-2]$ with an arrow pointing to the value '9102' in the list above.
 - At the bottom: $\text{dict}[\text{'Alice'}] =$

Accessing Values in Dictionary:

To access dictionary elements, you can use the familiar square brackets along with the key to obtain its value.

```
dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'};
print "dict['Name']: ", dict['Name'];
print "dict['Age']: ", dict['Age'];
```

If we attempt to access a data item with a key, which is not part of the dictionary, we get an error

Updating Dictionary:

You can update a dictionary by adding a new entry or item(i.e., a key-value pair), modifying an existing entry, or deleting an existing entry as shown below in the simple example:

```
dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'};

dict['Age'] = 8; # update existing entry
dict['School'] = "DPS School"; # Add new entry

print "dict['Age']: ", dict['Age'];
print "dict['School']: ", dict['School'];
```

Delete Dictionary Elements:

You can either remove individual dictionary elements or clear the entire contents of a dictionary. You can also delete entire dictionary in a single operation.

To explicitly remove an entire dictionary, just use the **del** statement.

Following is a simple example:

```
dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'};
del dict['Name']; # remove entry with key 'Name'
dict.clear(); # remove all entries in dict
del dict ; # delete entire dictionary
```

1, 2, 3

dict.appended.
dict.extend([4,5])

Properties of Dictionary Keys:

Dictionary values have no restrictions. They can be any arbitrary Python object, either standard objects or user defined objects. However, same is not true for the keys.

There are two important points to remember about dictionary keys:

(a) More than one entry per key not allowed. Which means no duplicate key is allowed. When duplicate keys encountered during assignment, the last assignment wins. Following is a simple example:

```
dict = {'Name': 'Zara', 'Age': 7, 'Name': 'Manni'};  
print "dict['Name']: ", dict['Name'];
```

(b) Keys must be immutable. Which means you can use string s, numbers or tuples as dictionary keys but something like ['key'] is not allowed. Following is a simple example:

```
dict = {'Name': 'Zara', 'Age': 7};  
print "dict['Name']: ", dict['Name'];
```


Built-in Dictionary Functions & Methods:

Python includes the following dictionary functions:

SN	Function with Description
1	<u>cmp(dict1, dict2)</u> Compares elements of both dict.
2	<u>len(dict)</u> Gives the total length of the dictionary. This would be equal to the number of items in the dictionary.
3	<u>str(dict)</u> Produces a printable string representation of a dictionary
4	<u>type(variable)</u> Returns the type of the passed variable. If passed variable is dictionary, then it would return a dictionary type.

dict.clear()

Removes all elements of dictionary *dict*

dict.copy()

Returns a shallow copy of dictionary *dict*

dict.fromkeys()

Create a new dictionary with keys from *seq* and values *set* to *value*.

dict.get(key, default=None)

For *key* key, returns value or default if key not in dictionary

a. = dict.pop("key") → return deleted value
del dict["key"]

dict.has_key(key)

Returns *true* if key in dictionary *dict*, *false* otherwise

dict.items()

dict.keys()

Returns list of dictionary *dict*'s keys

dict.setdefault(key, default=None)

Similar to *get()*, but will set *dict[key]=default* if *key* is not already in *dict*



[dict.keys\(\)](#)

Returns list of dictionary dict's keys

[dict.setdefault\(key, default=None\)](#)

Similar to `get()`, but will set `dict[key]=default` if *key* is not already in dict

What Are Python Modules?

- A **module** is a file containing Python definitions and statements.
- Modules help **organize code** and promote **reusability**.
- Python has many built-in modules (e.g., `math`, `random`, `statistics`).

$$(5, 8, 3, 2, 9, 11) \neq \underline{\text{mean}}.$$

Importing Modules

Using `import` Statement:

```
python
```

```
import math  
print(math.pi)
```

Using `from` Statement:

```
python
```

```
from math import pi  
print(pi)
```

- ✓ Use `import` to import the whole module.
- ✓ Use `from` to import specific functions or constants.

The `math` Module

- Provides mathematical functions.
- Commonly used constants and functions:

```
from math import pi, e, sqrt, ceil, floor, pow, fabs, sin, cos, tan
```

Function	Description
<code>pi</code>	Value of π
<code>e</code>	Euler's number
<code>sqrt()</code>	Square root
<code>ceil()</code>	Round up
<code>floor()</code>	Round down

Handwritten notes illustrating the behavior of `ceil` and `floor` functions:

- `2.68` is rounded up to `3` by `ceil`.
- `2.68` is rounded down to `2` by `floor`.

Trigonometric Functions

- Used for mathematical calculations involving angles.

python

```
from math import sin, cos, tan, pi
print(sin(pi/2)) # Output: 1.0
```

Random →

The random Module

- Used to generate random numbers.

python

```
import random

print(random.random()) # Float between 0 and 1
print(random.randint(1, 10)) # Integer between 1 and 10
print(random.randrange(1, 10, 2)) # Integer from range with step
```

The random Module

- Used to generate random numbers.

python

```
import random

print(random.random())      # Float between 0 and 1
print(random.randint(1, 10)) # Integer between 1 and 10
print(random.randrange(1, 10, 2)) # Integer from range with step
```

Function	Description
<code>random()</code>	Random float [0.0, 1.0)
<code>randint(a, b)</code>	Random int from a to b
<code>randrange(start, stop, step)</code>	Random int with step

0.2586

float

randrange(1, 8, 2)

The statistics Module

- Provides functions for statistical operations.

python

```
import statistics
```

```
data = [10, 20, 30, 40, 50]
print(statistics.mean(data))
print(statistics.median(data))
print(statistics.mode(data))
```

Function

Description

mean()

Arithmetic average

median()

Middle value

mode()

Most frequent value

⑤
31, 94, 35
34, 35, 94
—

— — — — —
↓
3rd place median. ⇒ 30

What is a loop in Python?

A loop in Python is a control flow statement that allows you to execute a piece of code repeatedly until a specific condition is met.

Loops in Python are necessary for operations that require repetitive execution, such as iterating through a Python list of items or doing calculations many times.

Different Types of Loops in Python

To understand the Loop Structures in Python, there are 3 Types of Loops in Python:

while loop
for loop
nested loop

→ conditions
→ sequence (initiator)

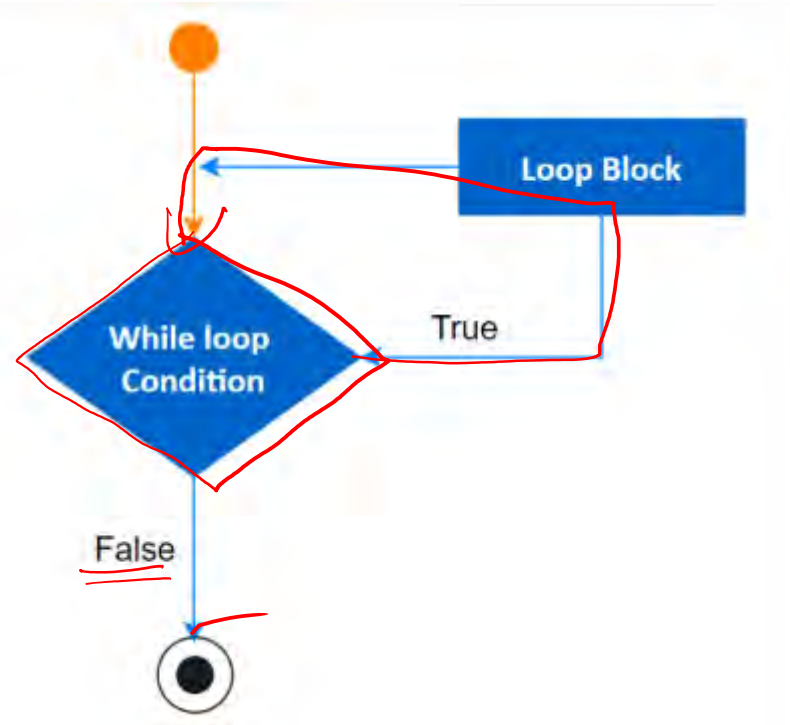
```
for (i=1, i <= 5, i++)  
{  
    _____  
    _____  
    _____  
}
```

Python while loop

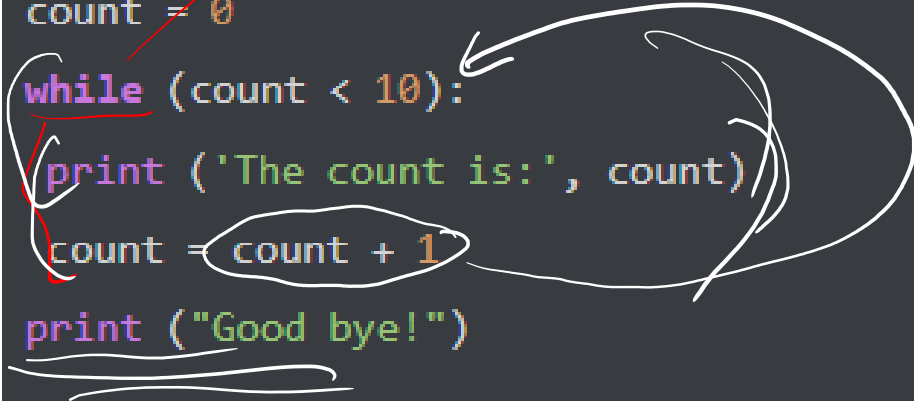
A while loop in Python is used to repeat a block of code as long as a certain condition is true.

It keeps running the loop body again and again until the condition becomes false.

This is useful when you don't know in advance how many times the loop should run.



```
count = 0  
while (count < 10):  
    print ('The count is:', count)  
    count = count + 1  
print ("Good bye!")
```



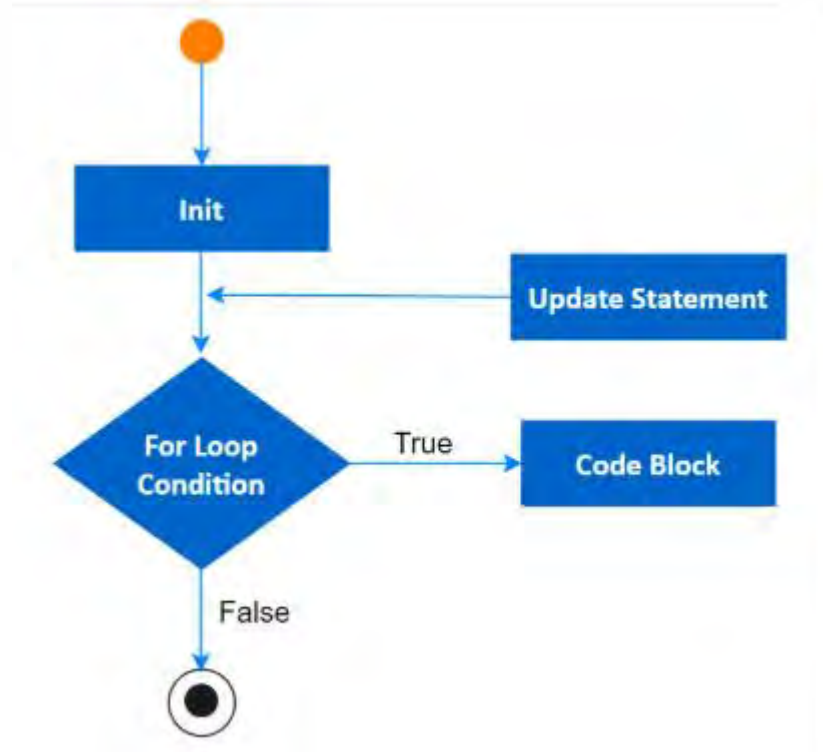
A while loop is a control flow statement that allows you to repeatedly run a block of code as long as a given condition is true.

Python for loop

A for loop in Python is used to go through a sequence of items, like a list, a string, or a range of numbers.

It repeats the block of code once for each item in the sequence.

This makes it very useful when you already know how many times you want to loop.



```
for letter in 'Python': # First Example
    print ('Current Letter :', letter)
fruits = ['guava', 'apple', 'mango']
for fruit in fruits: # Second Example
    print ('Current fruit :', fruit)
print ("Good bye!")
```

P
y
t
h
o
n

Python Nested loop

A nested loop in Python means using one loop inside another loop.

It is commonly used when you need to repeat actions in a multi-level structure, like printing patterns or working with multi-dimensional lists (like 2D arrays).

```
i = 2
while(i < 100):
    j = 2
    while(j <= (i/j)):
        if not(i%j): break
        j = j + 1
    if (j > i/j) : print (i, " is prime")
    i = i + 1
print ("Good bye!")
```


Loop Control Statements in Python

In Python, loop control statements are used to alter the regular flow of a loop.

They enable you to skip iterations, end the loop early, or do nothing at all.

Python's main three loop control statements are:

- 1-Break
- 2-Continue
- 3-Pass

Break

The break statement is used to quickly stop the loop regardless of whether the loop condition is still true.

This signifies that the loop will be terminated and the next statement outside of the loop will be executed.

```
for i in range(10):
```

```
    if i == 5:
```

```
        break
```

```
    print(i)
```

continue

This code in the Python Online Compiler prints the numbers 0 to 4, but then it exits the loop and moves on to the next statement.

Continue

The continue statement is used to skip the current loop iteration and go to the next iteration.

This implies that the code from the current iteration will not be performed, and the loop will continue to iterate until the loop condition is satisfied.

```
for i in range(10):  
    if i % 2 == 0:  
        continue  
    print(i)
```

1
3
5
7
9

This code generates the odd numbers 1 through 9. Any iterations where the integer is even are skipped by the continue statement.

break vs continue in Python Loops

Feature	break	continue
Purpose	Exits the loop entirely	Skips the current iteration only
Usage	When you want to stop the loop early	When you want to skip certain steps
Affects	Ends the loop and moves to next statement	Moves to the next loop iteration

Pass

The pass statement is a null statement that has no effect. It can be utilized as a placeholder in control statements or empty loops.

```
for i in range(10):  
    # Do nothing  
    pass
```

This code will not print anything because the pass statement does nothing. It is frequently used as a placeholder in control statements or empty loops.



for loop

✓ Use when **you know in advance** how many times you want to iterate (e.g., over a list, range, string, etc.).

while loop

✓ Use when you **don't know in advance** how many times to iterate and want to loop until a condition is false.

Print Even Numbers from 1 to 20

```
for i in range(2, 21, 2):  
    print(i)
```

Sum of Digits of a Number

```
num = 1234
sum_digits = 0
while num > 0:
    digit = num % 10
    sum_digits += digit
    num //= 10
```


Check if a Number is Prime

```
num = 17
is_prime = True

if num < 2:
    is_prime = False
else:
    for i in range(2, int(num**0.5)+1):
        if num % i == 0:
            is_prime = False
            break

print(f"{num} is Prime? {is_prime}")
```



2 mins Summary



TEACHING
WALLAH

Topic

while for loop }
nested loop

Dictionary ✓

Import modules x

THANK YOU